

Data Structures & Algorithms (CS-212)

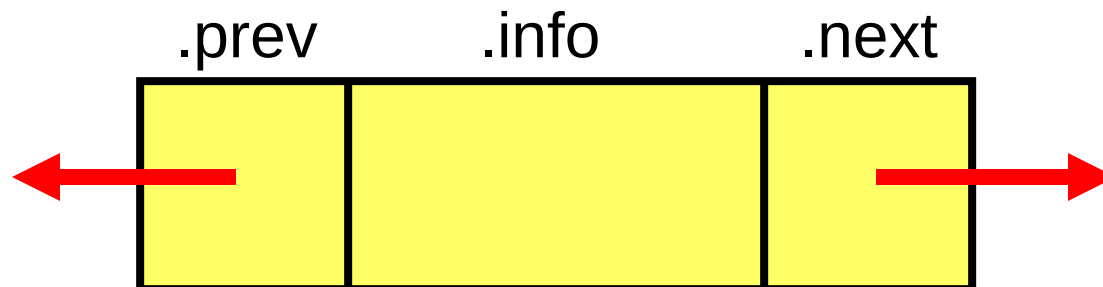
Doubly & Circularly Linked Lists

Singly Linked List

- Moving forward in a singly-linked list is easy; moving backwards is not so easy.
- To move back one node, we have to start at the head of the singly-linked list and move forward until the node before the current
- Time consuming to remove any element other than the head, because there is no quick way of accessing the element immediately preceding the one that is to be removed (which needs to be accessed so its next pointer value can be updated)

Doubly Linked List

- Stores
 - Element member
 - A next link
 - A previous link
- Allows for efficient insertion and removal at any point in the list



Sentinel Nodes

- **Head** and **tail** are **dummy** nodes, also called **sentinels**, do not store any **data** elements.
- **Head**: header sentinel previous reference (link) is null.
- **Tail**: trailer sentinel next reference (link) is null.

Header & Trailer Sentinels

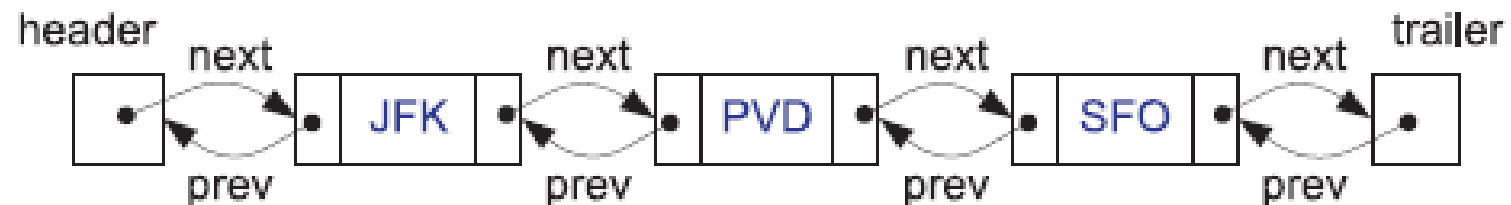


Figure 3.12: A doubly linked list with sentinels, *header* and *trailer*, marking the ends of the list. An empty list would have these sentinels pointing to each other. We do not show the null *prev* pointer for the *header* nor do we show the null *next* pointer for the *trailer*.

IMPLEMENTATION

A Simple Double Linked List Class

- We use two classes:
 - **DNode**
 - **DList**
- Declare DNode class for the nodes
 - data: double-type data in this example
 - next: a pointer to the next node in the list
 - prev: a pointer to the previous node in the list

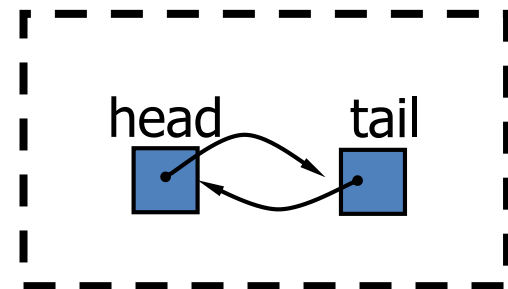
```
class DNode {  
public:  
    double data;           // data  
    DNode*   next;         // pointer to next  
    DNode*   prev;         // pointer to previous  
  
};
```

A Simple Double Linked List Class

- Declare `DList`, which contains
 - `head`: a pointer to the first node in the list.
Since the list is empty initially, `head` is set to `NULL`
 - `tail`: a pointer to the last node in the list. Since the list is empty initially, `tail` is set to `NULL`
 - Operations on `DList`

```
class DList {
public:
    DList(void) // constructor
    {
        head = new DNode();
        tail = new Dnode();
        head->next = tail;
        tail->prev= head;
    }
    ~DList(void);

private:
    DNode* head;
    Dnode* tail;
};
```



```
// destructor
```


A Simple Double Linked List Class

- Operations of `DList`
 - `IsEmpty`: determine whether or not the list is empty
 - `InsertNode`: insert a new node at a particular position
 - `FindNode`: find a node with a given value
 - `DeleteNode`: delete a node with a given value
 - `DisplayList`: print all the nodes in the list

A Simple Double Linked List Class

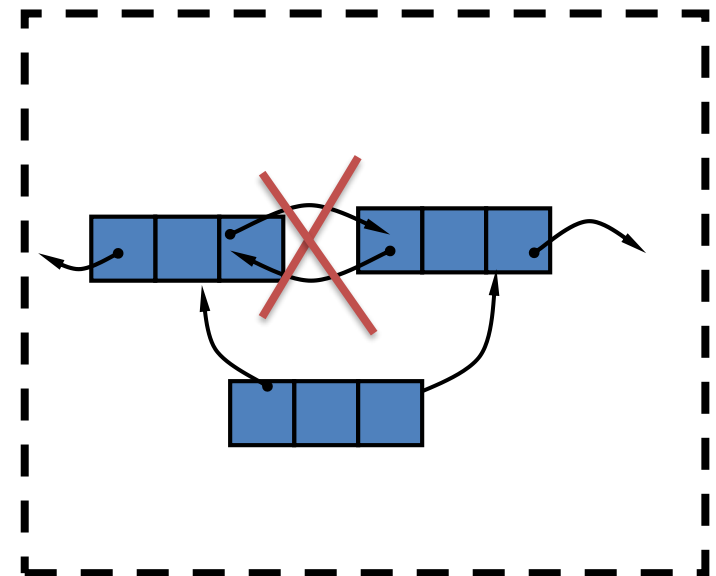
- Declare `DList`, which contains
 - `head`: a pointer to the first node in the list.
Since the list is empty initially, `head` is set to `NULL`
 - `tail`: a pointer to the last node in the list. Since the list is empty initially, `tail` is set to `NULL`
 - Operations on `DList`

```
class DList {
public:
    DList(void); // constructor
    ~DList(void); // destructor
    bool IsEmpty() { return (head->next == tail); }
    int FindNode(double x);
    void DisplayList(void);

private:
    DNode* head;
    Dnode* tail;
};
```

Inserting a new node

- `DNode* InsertNode(int index, double x)`
 - Insert a node with data equal to `x` after the `index`'th elements.
(i.e., when `index = 0`, insert the node as the first element;
when `index = 1`, insert the node after the first element, and so on)
 - If the insertion is successful, return the inserted node.
Otherwise, return `NULL`.
(If `index` is < 0 or $>$ length of the list, the insertion will fail.)
- Steps
 1. Locate `index`'th element
 2. Allocate memory for the new node
 3. Point the new node to its successor
 4. Point the new node's predecessor to the new node



Inserting a new node

- Possible cases of Insert a Node
 1. Insert into an empty list
 2. Insert in start
 3. Insert at the end
 4. Insert in middle

A Simple Double Linked List Class

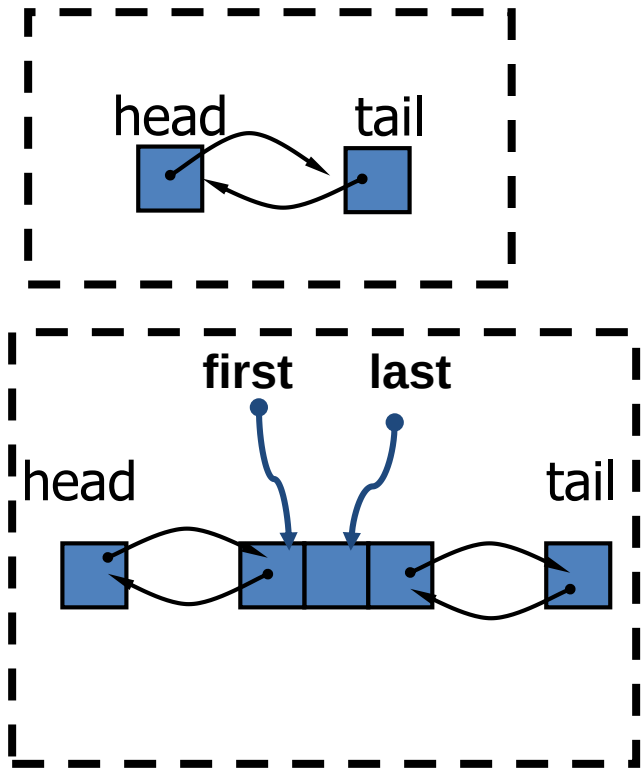
- Declare DList, which contains
 - head: a pointer to the first node in the list.
Since the list is empty initially, head is set to NULL
 - tail: a pointer to the last node in the list. Since the list is empty initially, tail is set to NULL
 - Operations on DList

```
class DList {
public:
    DList(void); // constructor
    ~DList(void); // destructor
    bool IsEmpty() { return (head->next == tail); }
    DNode* InsertNode(int index, double x);
    DNode* InsertAtStart(double x);
    DNode* InsertAtEnd(double x);
    int FindNode(double x);
    void DisplayList(void);

private:
    DNode* head;
    DNode* tail;
};
```

Insert into an empty list

- This single node is the first node, and also is the last node:
- Steps
 1. Create node
 2. Set the node data value
 3. Set pointers
 1. New node next points to tail node
 2. New node prev points to head node
 3. Head next points to new node
 4. Tail prev points to new node



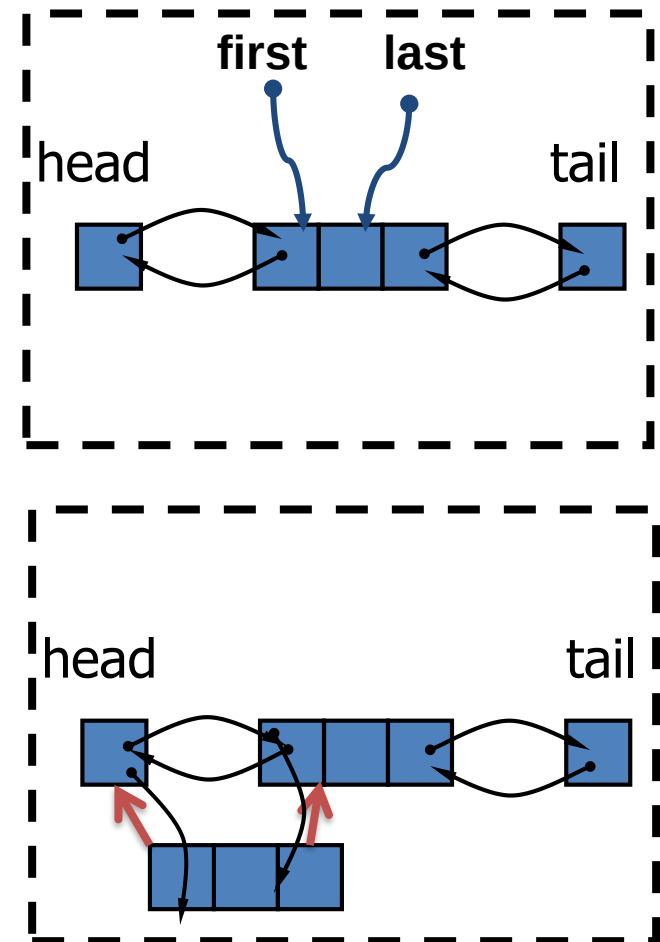
Inserting a new node

- Possible cases of Insert a Node
 1. Insert into an empty list
 2. Insert in start
 3. Insert at the end
 4. Insert in middle

Insertion at the start

Steps:

- Create a Node
- Set the node data Values
- Set pointers
 1. New node next points to head next node
 2. New node prev points to head node
 3. Head next points to new node
 4. New node->next-> prev points to new node



Implementation InsertAtStart

```
DNode* InsertAtStart(double x)
{
    DNode* newNode = new DNode();
    newNode->data = x;
    newNode->next = head->next;
    newNode->prev = head;
    head->next = newNode;
    newNode->next->prev = newNode;
    return newNode;
}
```

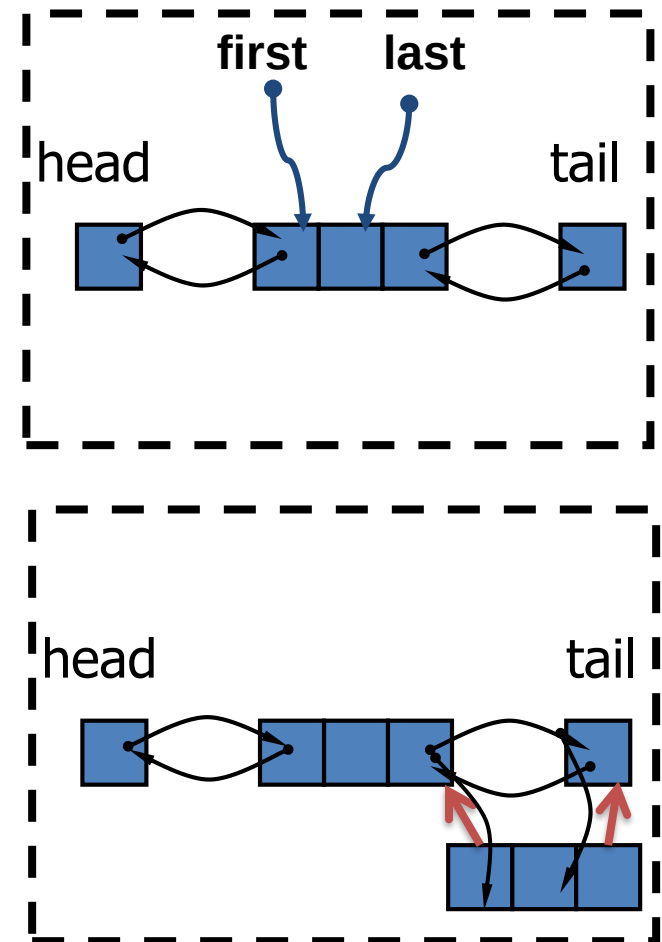
Inserting a new node

- Possible cases of Insert a Node
 1. Insert into an empty list
 2. Insert in start
 3. Insert at the end
 4. Insert in middle

Insertion at the end

Steps:

- Create a Node
- Set the node data Values
- Set pointers
 1. New node next points to tail node
 2. New node prev points to tail->prev node
 3. Tail->prev->next points to new node
 4. Tail->prev points to new node



Implementation Insert at the end

```
DNode* InsertAtEnd(double x)
{
    DNode* newNode = new DNode();
    newNode->data = x;
    newNode->next = tail;
    newNode->prev = tail->prv;
    tail->prev->next = newNode;
    tail->prev = newNode;
    return newNode;
}
```

Inserting a new node

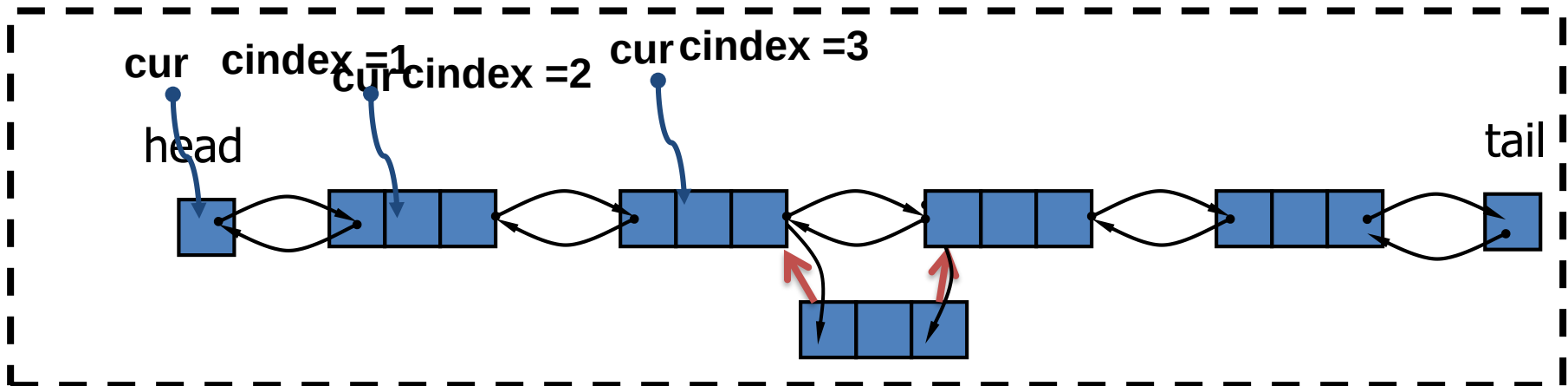
- Possible cases of Insert a Node
 1. Insert into an empty list
 2. Insert in start
 3. Insert at the end
 4. Insert in middle

Insertion in middle

Steps:

- Locate the index
- Create a Node
- Set the node data Values
- Set pointers
 1. New node next points to current node next
 2. New node prev points to current node
 3. cur node next points to new node
 4. Cur node->next ->prev points to new node

index =3



Delete a node

- Delete from start
- Delete from tail
- Delete from middle

A Simple Double Linked List Class

- Declare `DList`, which contains
 - `head`: a pointer to the first node in the list.
Since the list is empty initially, `head` is set to `NULL`
 - `tail`: a pointer to the last node in the list. Since the list is empty initially, `tail` is set to `NULL`
 - Operations on `DList`

```
class DList {
public:
    DList(void); // constructor
    ~DList(void); // destructor
    bool IsEmpty() { return (head->next == tail); }
    DNode* InsertNode(int index, double x);
    DNode* InsertAtStart(double x);
    DNode* InsertAtEnd(double x);
    int DeleteFromStart(double x);
    int DeleteFromEnd(double x);
    int DeleteElement(double x);
    int FindNode(double x);
    void DisplayList(void);

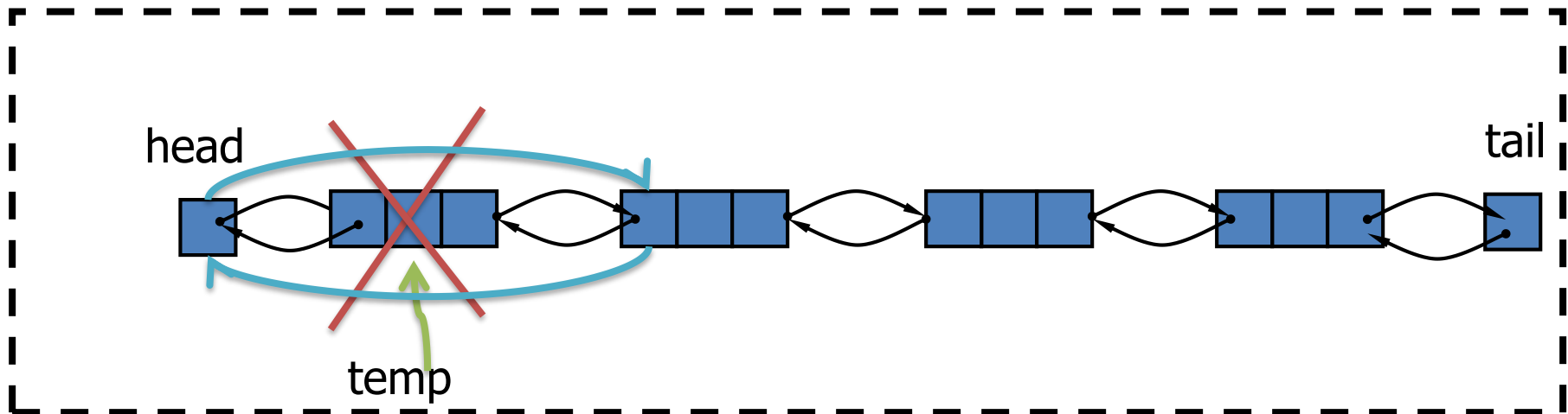
private:
    DNode* head;
    DNode* tail;
};
```


Delete a node

- Delete from start
- Delete from tail
- Delete from middle

Delete from the start

- Steps
 - Create a temporary node and points to the head->next
 - head->next points to temp->next
 - temp->next->previous points to head
 - Release memory



Implementation

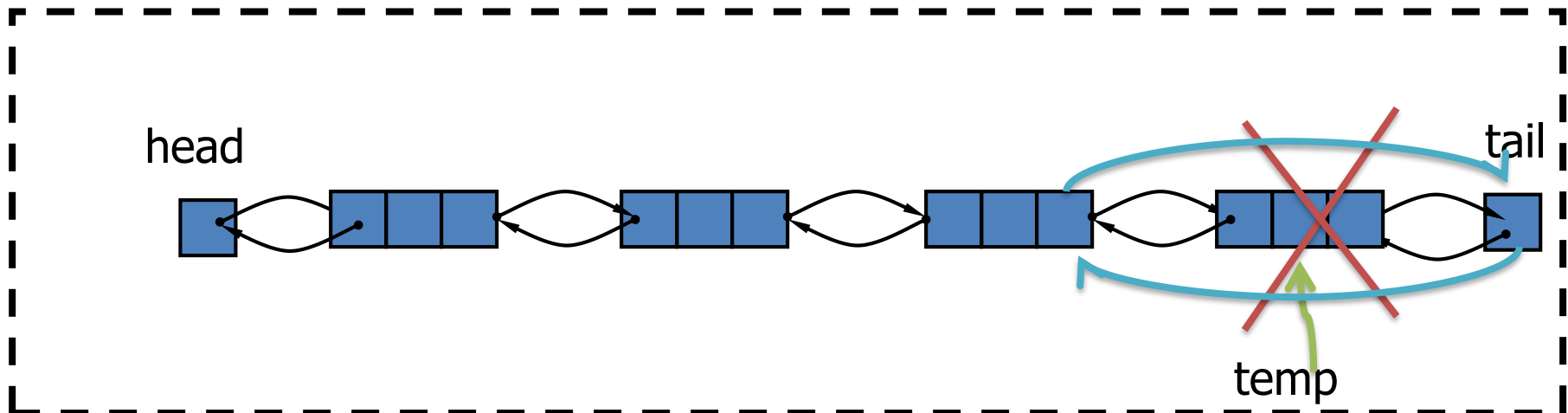
```
int DeleteFromStart (double x)
{
    DNode* tempnode = head->next;
    head->next = tempnode->next;
    tempnode->next->prev = head;
    delete tempnode;
    return 1;
}
```

Delete a node

- Delete from start
- Delete from tail
- Delete from middle

Delete from tail

- Steps
 - Create a temporary node and points to the tail->prev
 - tail->prev points to temp->prev
 - temp->prev->next points to tail
 - Release memory



Implementation

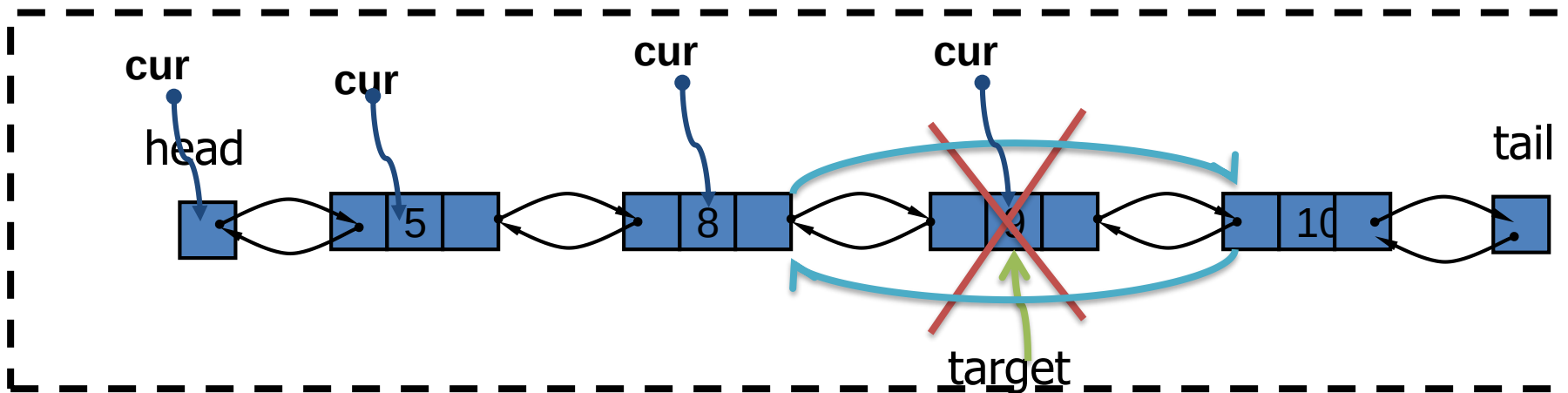
```
int DeleteFromEnd(double x)
{
    DNode* tempnode = tail->prev;
    tail->prev = tempnode->prev;
    tempnode->prev->next = tail;
    delete tempnode;
    return 1;
}
```

Delete a node

- Delete from start
- Delete from tail
- Delete from middle

Delete from middle

- Steps
 - Create a temporary node and points to the head
 - Locate node index
 - Create another temp node(target) and points to cur
 - $\text{target} \rightarrow \text{next} \rightarrow \text{prev}$ points to $\text{target} \rightarrow \text{prev}$
 - $\text{target} \rightarrow \text{prev} \rightarrow \text{next}$ points to $\text{target} \rightarrow \text{next}$
 - Release memory



Implementation

```
int DeleteElement(double x)
{
    DNode* cur = head;
    while(cur->next!=tail && cur->data!=x)
    {
        cur = cur->next;
    }
    if(cur->data==x)
    {
        DNode* target = cur;
        target->next->prev = target->prev;
        target->prev->next = target->next;
        delete target;
        cur = NULL;
        return 1;
    }
    else
        return 0;
}
```

Finding a node

- `int FindNode(double x)`
 - Search for a node with the value equal to `x` in the list.
 - If such a node is found, return its position. Otherwise, return 0.

```
int List::FindNode(double x) {  
    DNode* currNode    =    head;  
    int currIndex =    1;  
    while (currNode && currNode->data != x) {  
        currNode    =    currNode->next;  
        currIndex++;  
    }  
    if (currNode) return currIndex;  
    return 0;  
}
```

Printing all the elements

- `void DisplayList(void)`
 - Print the data of all the elements
 - Print the number of the nodes in the list

```
void DList::DisplayList()
{
    int num          = 0;
    DNode* currNode  = head;
    while (currNode != NULL) {
        cout << currNode->data << endl;
        currNode      = currNode->next;
        num++;
    }
    cout << "Number of nodes in the list: " << num << endl;
}
```

Printing all the elements in reverse

```
int num          =      0;
DNode* currNode  =      tail;
while (currNode != NULL) {
    cout << currNode->data << endl;
    currNode      =      currNode->prev;
    num++;
}
```

Destroying the list

- `~DList(void)`
 - Use the destructor to release all the memory used by the list.
 - Step through the list and delete each node one by one.

```
DList::~~DList(void) {  
    DNode* currNode = head, *nextNode = NULL;  
    while (currNode != NULL)  
    {  
        nextNode      =      currNode->next;  
        // destroy the current node  
        delete currNode;  
        currNode      =      nextNode;  
    }  
}
```

CIRCULAR LINKED LIST

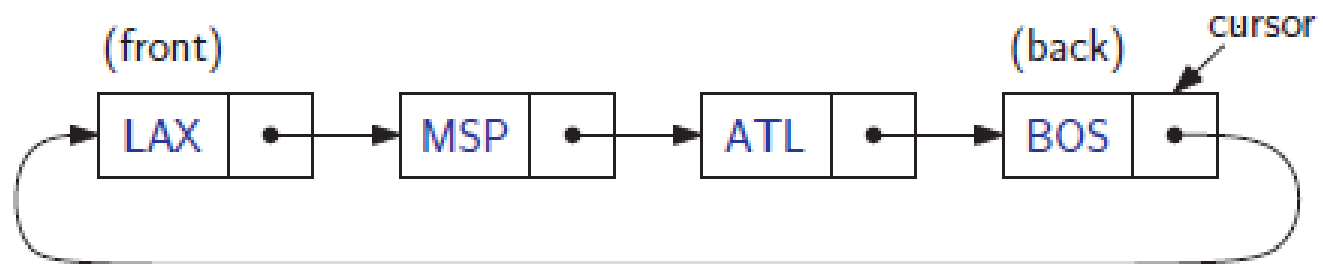


Figure 3.15: A circularly linked list. The node referenced by the cursor is called the back, and the node immediately following is called the front.

```
typedef string Elem;           // element type
class CNode {                  // circularly linked list node
private:
    Elem elem;                 // linked list element value
    CNode* next;               // next item in the list

    friend class CircleList;   // provide CircleList access
};
```

Code Fragment 3.28: A node of a circularly linked list.


```

class CircleList {                                // a circularly linked list
public:
    CircleList();                                // constructor
    ~CircleList();                               // destructor
    bool empty() const;                          // is list empty?
    const Elem& front() const;                   // element at cursor
    const Elem& back() const;                   // element following cursor
    void advance();                              // advance cursor
    void add(const Elem& e);                     // add after cursor
    void remove();                              // remove node after cursor
private:
    CNode* cursor;                              // the cursor
};

```

Code Fragment 3.29: Implementation of a circularly linked list class.

```
CircleList::CircleList()           // constructor
    : cursor(NULL) { }
CircleList::~CircleList()          // destructor
    { while (!empty()) remove(); }
```

Code Fragment 3.30: The constructor and destructor.

```

bool CircleList::empty() const           // is list empty?
{ return cursor == NULL; }
const Elem& CircleList::back() const     // element at cursor
{ return cursor->elem; }
const Elem& CircleList::front() const    // element following cursor
{ return cursor->next->elem; }
void CircleList::advance()               // advance cursor
{ cursor = cursor->next; }

```

Code Fragment 3.31: Simple member functions.

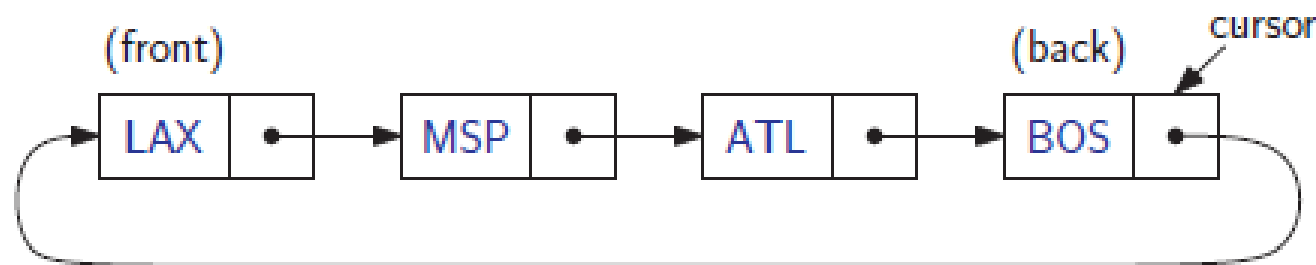


Figure 3.15: A circularly linked list. The node referenced by the cursor is called the back, and the node immediately following is called the front.

```

void CircleList::add(const Elem& e) {           // add after cursor
    CNode* v = new CNode;                      // create a new node
    v->elem = e;
    if (cursor == NULL) {                      // list is empty?
        v->next = v;                          // v points to itself
        cursor = v;                          // cursor points to v
    }
    else {                                     // list is nonempty?
        v->next = cursor->next;               // link in v after cursor
        cursor->next = v;
    }
}

```

Code Fragment 3.32: Inserting a node just after the cursor of a circularly linked list.

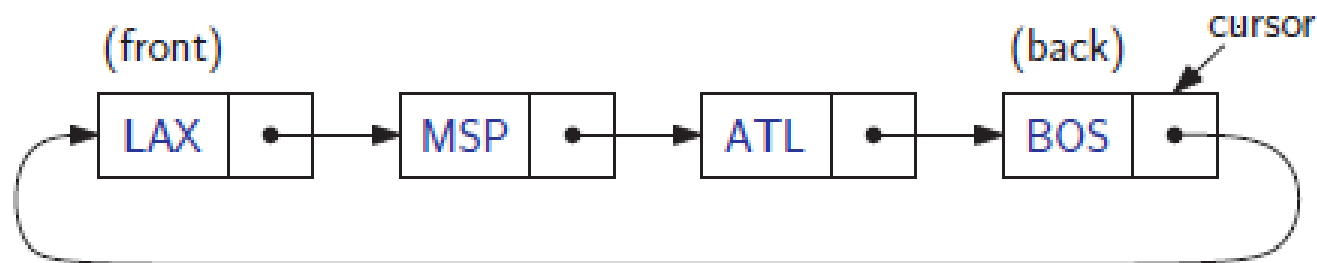


Figure 3.15: A circularly linked list. The node referenced by the cursor is called the back, and the node immediately following is called the front.

```
void CircleList::remove() {           // remove node after cursor
    CNode* old = cursor->next;         // the node being removed
    if (old == cursor)                 // removing the only node?
        cursor = NULL;                // list is now empty
    else
        cursor->next = old->next;       // link out the old node
    delete old;                        // delete the old node
}
```

Code Fragment 3.33: Removing the node following the cursor.

```

int main() {
    CircleList playList;           // []
    playList.add("Stayin Alive");  // [Stayin Alive*]
    playList.add("Le Freak");      // [Le Freak, Stayin Alive*]
    playList.add("Jive Talkin");   // [Jive Talkin, Le Freak, Stayin Alive*]

    playList.advance();            // [Le Freak, Stayin Alive, Jive Talkin*]
    playList.advance();            // [Stayin Alive, Jive Talkin, Le Freak*]
    playList.remove();             // [Jive Talkin, Le Freak*]
    playList.add("Disco Inferno"); // [Disco Inferno, Jive Talkin, Le Freak*]
    return EXIT_SUCCESS;
}

```

Code Fragment 3.34: Using the CircleList class to implement a playlist for a digital audio player.

Use of Doubly Linked List

- Doubly linked lists are useful for playing video and sound files with “rewind” and “instant replay”
- They are also useful for other linked data which require “rewind” and “fast forward” of the data

Lecture content adapted from Michael T. Goodrich textbook,
chapters 3.